1	2	3	$\sum$

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 02

Release: 22.10.2018; Due: 02.11.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Sparse Polynomials (__/10)

In class, you saw two ways to represent a polynomial of degree n. The coefficient representation becomes inefficient if many coefficients of the polynomial are 0 *(such polynomials are called* sparse*). Develop an efficient representation for sparse polynomials and describe algorithms for addition, multiplication, and evaluation in this representation. Analyse the efficiency of your algorithms in dependence of the degree n of the polynomials and the number k of nonzero coefficients.*

Extra credit (5 Points)*: Implement your data structure in Java or Python.*

Unserer Vorschlag wäre einen linked List zu verwenden.

In unserem linked List werden wir 2 Variablen haben und ein Zeiger auf dem nächsten Element.

Die Knoten werden wir in absteigender Reihenfolge anordnen, d.h. erstmal die Noden mit dem höheren Potenz, dann die mit kleinerem Potenz.

Wir werden die Knoten so bauen :

```
struct Node polynomteils
{
int koeffizient;
int exponent;
struct Node *next;
};


struct Node verwaltung
head *polynomteils current *polynombetr tail *polynomteils }
```

Damit können wir sowohl Speicherplatz sparen, als auch die Koffizienten 0 vermeiden, weil wir sowohl den Exponent speichern als auch den Zeiger auf den nächsten Element,das auch ein kleineren Exponent als (k-1) haben kann.

Addition (Pseudocode:

2 Sorting Experiments (__/10)

In "Imperative and Object Oriented Programming", you saw several algorithms for sorting an array of n numbers. Choose three of them, and implement them in Java or Python. Among your algorithms, there should be one with worst-case running time $\Theta(n^2)$ and one with worst-case running time $\Theta(n \log n)$. Test your implementations with random inputs and describe for each algorithm a class of inputs that represents an "extreme" case.
Measure the number of milliseconds that your implementations need for inputs of at least one hundred different sizes. For each input size, pick at least ten random inputs.
Discuss your results. What are the advantages and disadvantages of the respective algorithms? Which algorithm would you use under which circumstances? What do your experiments tell you about the O-Notation?

MergeSort :
Funktioniert am schnellsten,bei großen Eingaben braucht nur die Hälfte der Zeit die die anderen brauchen.
Dies wird auch bei der O-Notation erläutert,als dieser Algorithm den schnellsten Laufzeit hat und zwar $O(n \log n)$
Fast keine Vergleichen am Anfang,weil die Listen in kleineren Listen geteilt (divide and conquer). Danach aber muss man mehr bei der Mergefunktion vergleichen,damit die Listen in der richtigen Reihenfolge gebracht werden können.Trotzdem,wenn die Listen verschiedene Größe haben, können einige Vergleiche am Ende vermeidet werden. Der Algorithmus hat eine Durchnittliche TimeLapse bei unseren Eingaben von 1.21... Sekunden,fast doppelt mehr als Mergesort.Ich würde MergeSort verwenden, wenn ich mit großen Mengen an Daten arbeite und keine Speicherplatzbegrenzungen habe,weil der nicht "in Place" sortiert.

InsertionSort :
Gehört zu der $O(n^2)$,Nicht so schnell wie MergeSort, aber doch schneller als BubbleSort, obwohl, dass beiden die selbe O-Notation Komplexität haben. Wahrscheinlich ist dies auch von der Eingabe abhängig,weil BubbleSort vom hinten nach vorne sortiert,als Insertionsort dies vom vorne nach hinten macht. So wenn die Liste "fast" sortiert ist,wird Bubblesort wahrscheinlich dies schneller sortieren, wenn aber nicht,muss man mehreren Elementenplätze bei BubbleSort als bei InsertionSort tauschen, deswegen auch ist InsertionSort schneller und nicht so naiv wie QuickSort. Er hat eine Durchnittliche TimeLapse bei unseren Eingaben von 2.21... Sekunden,fast doppelt mehr als Mergesort.Ich würde Insertionsort verwenden,wenn ich entweder Speicherbegrenzungen hätte,oder mit kleineren Mengen an Daten arbeite.

BubbleSort :
Wie oben gesagt ist der Algorithmus langsamer als die anderen zwei. Er gehört zu der $O(n^2)$,aber ist langsamer als InsertionSort bei bestimmten Eingaben (wie oben erläutert).Im Vergleich zu MergeSort ist BubbleSort auch bis zu 3 Mal langsamer,was ein ziemlich großen Unterschied macht,wenn man mit großen Mengen an Daten arbeitet. Es hat eine Durchnittliche TimeLapse bei unseren Eingaben von 3.01... Sekunden,fast dreimal mehr als Mergesort.Ich würde BubbleSort verwenden,wenn ich die Logikfehler vermeiden will,oder wenn ich nach einer leichteren Implementierung suche, man muss in Kauf aber nehmen,dass er für großen Mengen an Daten nicht geeignet ist.
Unsere Experimenten haben uns gezeigt,dass die O-Notation ein ziemlich sicherliches Kriterium für die Bestimmung von der Laufzeit von Algorithmen ist, obwohl dass es auch andere Faktoren wie Eingabedaten und Speicherbedarf gibt. Mann kann sicher sein, dass

die O-Notation bei großen Eingaben eine sehr wichtige Rolle spielt, weil da nicht mehr um milisekunden,sondern um Sekunden geht und dies ist schon für den Benutzer sichtbar. Die Komplexität der Algorithmen kann nie mit festen Zahlen bestimmt werden (wegen verschiedener Eingaben), aber mann muss immer die O-Notation in Kauf nehmen, weil es sicherlich eine sinvolle Anordnung dazu ist.

3 The Towers of Hollywood (__/10)

You probably know the legend of the towers of Hanoi that was invented by the French mathematician Édouard Lucas in order to sell a wooden puzzle.
In the great temple of Benares, there is a wooden plate on which there are three vertical rods made out of diamond. On these rods, there are n disks made of pure gold, of n different sizes. At the beginning of time, the god Brahma has put the disks onto the first rod, sorted according to their size, such that the largest disk lies at the bottom. The Brahmins in the temple work tirelessly and transport the disks from rod to rod, obeying the following rules: (i) in each step, at most one disk can be moved from one rod to another; and (ii) a larger disk may never lie on top of a smaller disk. As soon as all the disks have reached the final rod, the world will end.

(a) *Describe a recursive algorithm that allows the Brahmins to reach the end of the world quickly. Determine exactly the number of moves that are necessary for n disks. How many years do we need to wait for the end of the world, assuming that Brahma dropped 100 disks onto the first rod and that it takes one day to move a single disk?*
So wir gehen davon aus,dass die Turmen mit A, B und C wie auf dem Bild gennant sind. Wir werden die Pseudocode nur mit wörter beschreiben: Als Eingabe werden wir die Anzahl n an den Ringen verlangen.
Wir haben die Bediengung am Anfang,dass die "rods" nur 3 sind,deswegen werden wir der Algorithmus für 3 beschreiben. Unser Meinung nach gehört der Algorithmus zu der Komplexitätsklasse von $O(2^n)$
Wir betrachten wie sich die Rekursion entwickelt und zwar:
$T(n) = 1 + 2 + 2^2 + ... + 2^{n-1}...$
$T(n) = 2 * T(n-1) + 1$
Wir bilden den charakteristischen Polynom:
$x^n - 2 * x^{n-1} = 0$
Damit:
$T(n) = c_1 * 2^n + c_2$
Wir setzen ein:
$T(0) = 0$
$T(1) = 1$
Damit auch:
$c_1 + c_2 = 0$
$2 * c_1 + c_2 = 1$
Damit $c_2 = -c_1 = -1$ und damit auch:
$T(n) = 2^n - 1$
Wenn mann dies in Tagen betrachtet würde das :
1267650600228229401496703205375 Tagen mit 100 Disks dauern.

(b) *Recently, the temple has moved to California, where the Brahmins have adopted a much more relaxed attitude. The disks are now made out of gilded cardboard, and the rods are plastic. Most importantly, the rules for the disk ordering have been relaxed: we still require that on each rod the largest disk lie at the bottom, but the order of the other disks is arbitrary. In the end, all disks again must reach the last rod in the correct order, largest to smallest from bottom to top.*
Describe an efficient algorithm for the relaxed problem. Determine the exact number of moves your algorithm requires for n disks. How many years do we need to wait now for the end of the world, assuming that Brahma dropped 100 disks onto the first rod and that it takes one day to move a single disk?
Wir haben der Algorithmus in Python implementiert.
Also wie wir die Anzahl der Schritten bestimmt haben ist ja sehr einfach.
Zu jedem nth disk brauchen wir genau n "moves",damit alle disks auf ihn liegen.
Damit können wir ein "rod" freilassen,damit den $n+1$th disk danach da aufgelegt wird.
Wir müssen nur betrachten,dass wir bei dem letzten "disk" nur einen Schritt brauchen.
Danach muss man die disks neu anordnen.Dafür braucht man auch für den n-ten disk $n-1$ Bewegunden,damit die disks,die oben liegen zu entfernen (hier ist es wichtig zu merken,dass der nächstgrößeren Disk immer ganz oben sitzt,also der n-1th Disk) und 1 Schritt,damit der n th disk auf dem $n+1$ aufgelegt wird.
Damit können wir die Schritte leicht mit dem Gaußsumme berechnen und zwar:
$\frac{n(n-1)}{2} * 2 + 1$,weil wir für den letzten Disk nur 1 Schritt brauchen,addieren wir 1 und wir nehmen die Gaußsumme von $n-1$ Disks.Wir multiplizieren dies mit 2,weil wir das einmal machen,bis zu dem Zeitpunkt,wenn wir den letzten Disk erreicht haben und dann nochmal,damit wir die anderen auf dem in der richtigen Reihenfolge bringen können (wie oben erläutert).
Damit würde es bei 100 Disks:
$100 * 99 + 1 = 9901$ Tagen dauern und dies ist offensichtlich weniger als die exponentielle Laufzeit des ursprunglichen Problems.